

SE 311 Homework Assignment 1

Overview

In this assignment, you will design and implement a **Key Word in Context (KWIC)** system. The system accepts a plain text (`.txt`) file containing a set of sentences and supports multiple text-processing and indexing functions.

The primary goal of this assignment is to practice **data design, modular decomposition, and decision analysis**, while explicitly considering how the design can accommodate future requirement changes.

Your implementation must be written in **Java** and accessed through **console-based commands**.

Functional Requirements

1. KWIC Processing

Given an input file containing multiple sentences, the system shall:

- Generate all **circular shifts** of each sentence.
- Output all circularly shifted sentences in **alphabetical order**.
- Append the **original line number** to each shifted line.

Input (Original Lines):

"1: Sense and Sensibility"

"2: Software Architecture"

"3: Crouching Tiger Hidden Dragon"

Sample Output:

Index | Circular Shifted Lines | Original Line Index"

"1 | and Sensibility Sense | 1"

"2 | Architecture Software | 2"

"3 | Crouching Tiger Hidden Dragon | 3"

"4 | Dragon Crouching Tiger Hidden | 3"

"5 | Hidden Dragon Crouching Tiger | 3"

"6 | Sense and Sensibility | 1"

"7 | Sensibility Sense and | 1"

"8 | Software Architecture | 2"

"9 | Tiger Hidden Dragon Crouching | 3"

2. Keyword Search

The system shall support interactive keyword search:

- The user enters a keyword via the console.
- The system returns **all original sentences** containing the keyword.
- In each returned sentence, the keyword must be **highlighted**.
- The system also reports the **total number of lines** containing the keyword.

Sample Output:

"1 sentence is found:"

"1 Crouching **Tiger** Hidden Dragan"

If the keyword does not appear in any sentence, the system shall output:

"[keyword] not found."

3. Index Generation

Given the list of input sentences, the system shall generate a **keyword index** with the following properties:

- All keywords are listed in **alphabetical order**.
- Each keyword is followed by the **line number(s)** of the original sentence(s) in which it appears.

- If a keyword appears in multiple lines, **all corresponding line numbers** must be listed.

Sample Output:

"and, 1"

"Architecture, 2"

"Crouching, 3"

"Dragon, 3"

"Hidden, 3"

"Sense, 1"

"Sensibility, 1"

"Software, 2"

"Tiger, 3"

4. User Interaction

All system functions must be accessible through **console commands**, allowing the user to:

- Run KWIC processing
- Perform keyword search
- Generate the keyword index

We should be able to run your program as follows:

Program entrypoint: `Main`

Launch: `java -cp out Main inputs\sample.txt`

Results printed to `STDOUT`

Supported commands (examples):

- `kwic` → prints KWIC table (one sentence per line)
 - `search <keyword>` → prints matching original lines with keyword highlighted
 - `index` → prints keyword index (one keyword per line)
 - `quit`
-

Possible Future Changes

Your design should explicitly consider the following possible future changes. You do **not** need to implement these changes in this assignment, but your design should make them feasible.

1. Input Format Changes

- Current: plain `.txt` files
- Future: `.csv` or other structured file formats

2. Index Generation Policy Changes

- Current: index all keywords
- Future: allow filtering of stop words (e.g., “a”, “an”, “the”, “and”)

3. Alphabetizing Policy Changes

- Current: any fixed sorting policy
- Future: user-selectable policies (ascending, descending, case-sensitive, case-insensitive)

4. Output Method Changes

- Current: console output
 - Future: output to files or web pages, possibly with different formats
-

Submission Guidelines

Submit **one folder** that can be opened directly in **Visual Studio Code (VS Code)**. The folder must include the following items.

Item 1: Data Design (Markdown)

- Submit a **.md file** documenting your data design decisions by answering the following questions:
 - What data structure did you use to store the lines read from the input file?
 - What data structure did you use to store the circularly shifted lines (or their indexes)?
 - What data structure did you use to store the sorted lines (or their indexes)?

Item 2: Module Design (UML Class Diagram)

- Submit a **PlantUML .md file** that renders correctly.
- **Do not reverse-engineer** the UML diagram from the source code.
- The UML diagram must clearly illustrate:
 - Major classes
 - Key attributes and methods
 - Relationships among modules
- Explain how your design will accommodate the possible changes listed above.

Item 3: Logic and Naming Design (Java Source Code)

- Submit a **Java project folder** that:
 - Can be opened and run in VSCode **without modification**
 - Includes all required source files under the **./src** directory
- Include a **README.md** file explaining:

- How to compile the program
 - How to run the program
 - How to use the console commands
-

Basic Naming Conventions

Your implementation must follow these conventions:

- **Class names** begin with capital letters
 - Example: `KeywordIndex`
- **Method names** start with a verb
 - Example: `calculateWeight()`
- **Variable names** begin with lowercase letters
 - Example: `lineCount`
- **Constant names** use all capital letters with underscores
 - Example: `I_AM_A_CONSTANT`
- **Accessor methods** use standard prefixes:
 - `getX()`, `setX()`, `isX()` (for Boolean values)
- **Parameter naming**
 - One acceptable convention is to use the prefix `a`, for example:

`sum(int anInteger1, int anInteger2)`
- **Files, classes, and methods** should be named to reflect **domain concepts** or **functional responsibilities**, not implementation details.